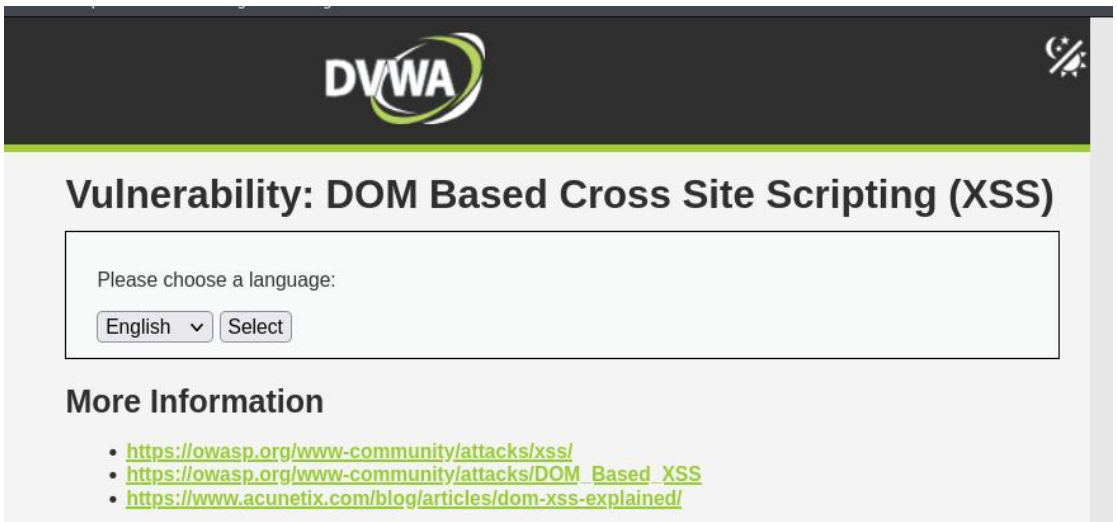
DOM-Based XSS (type-0 XSS) — DVWA



DOM-Based XSS — DVWA

- **Cross-Site Scripting (XSS):**
  It is a vuln that allows attackers to inject malicious client-side scripts (JS) into web pages viewed by other users. These scripts (Bypass access controls, Hijack usr sessions, Steal cookies/tokens, Deface websites)

- **Document Object Model (DOM):**
  It is a programming interface that represents an HTML doc as a hierarchical tree structure, allowing programs to access and modify content dynamically

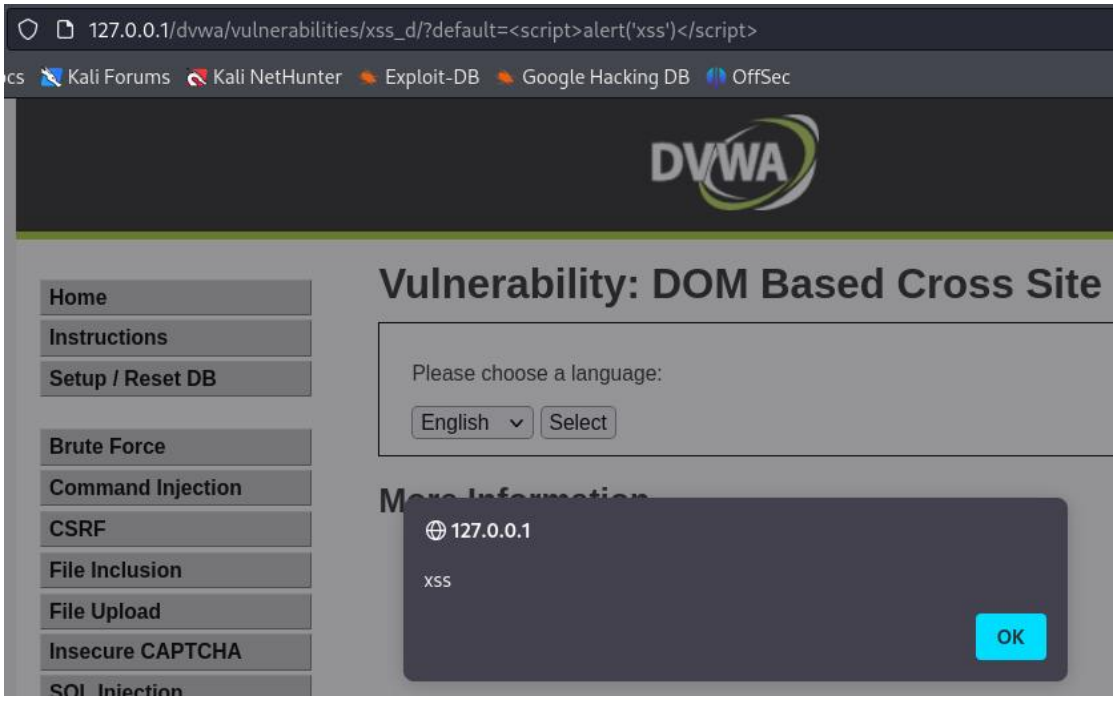
- **Type-0 XSS:**
  It runs on the client-side, where a JS modifies the DOM using

untrusted input (URL parameters), so the server never sees the attack, and the browser executes it

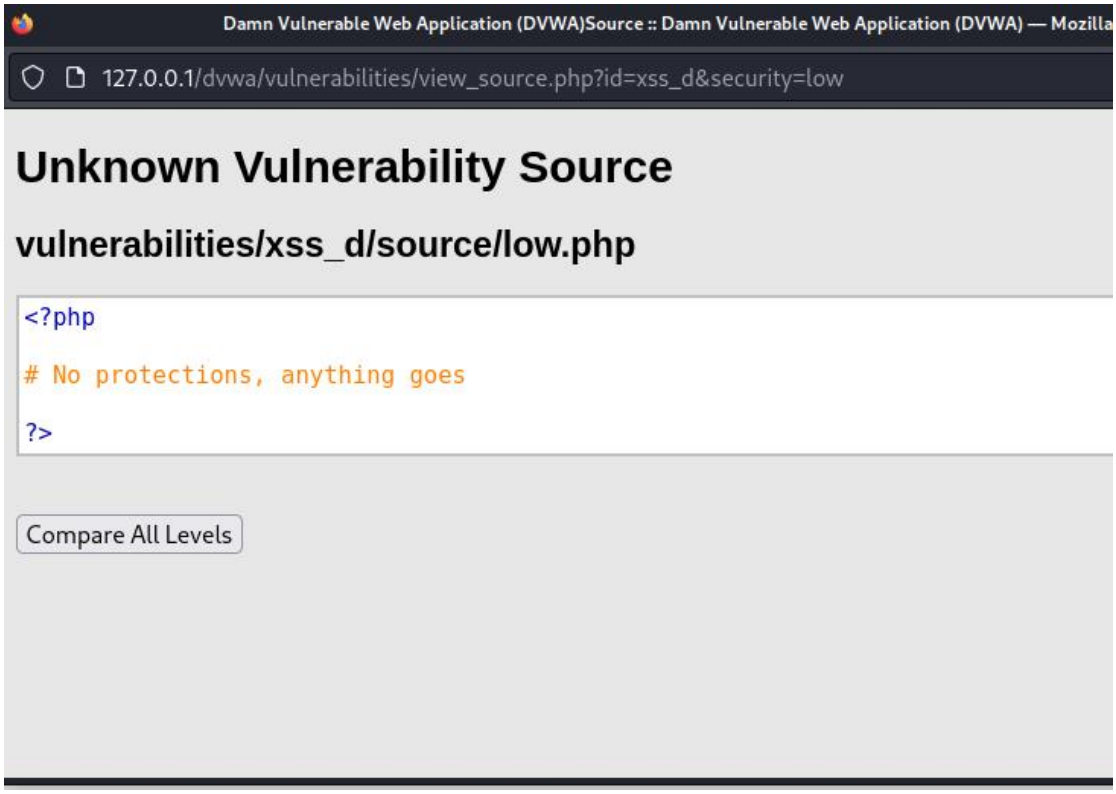
- **To run DVWA:**
  I- Web server (shows the DVWA pages in the browser)
  II- PHP (runs the DVWA code)
  III- MySQL DB (stores users, sessions, and data)
  IV- DVWA config (connects PHP to the database)

_ Here we go to solve type-0 XSS:

I- Low Security Level: No input validation is applied, which makes all vulnerabilities easy to exploit.



type-0 XSS — Low sec lvl



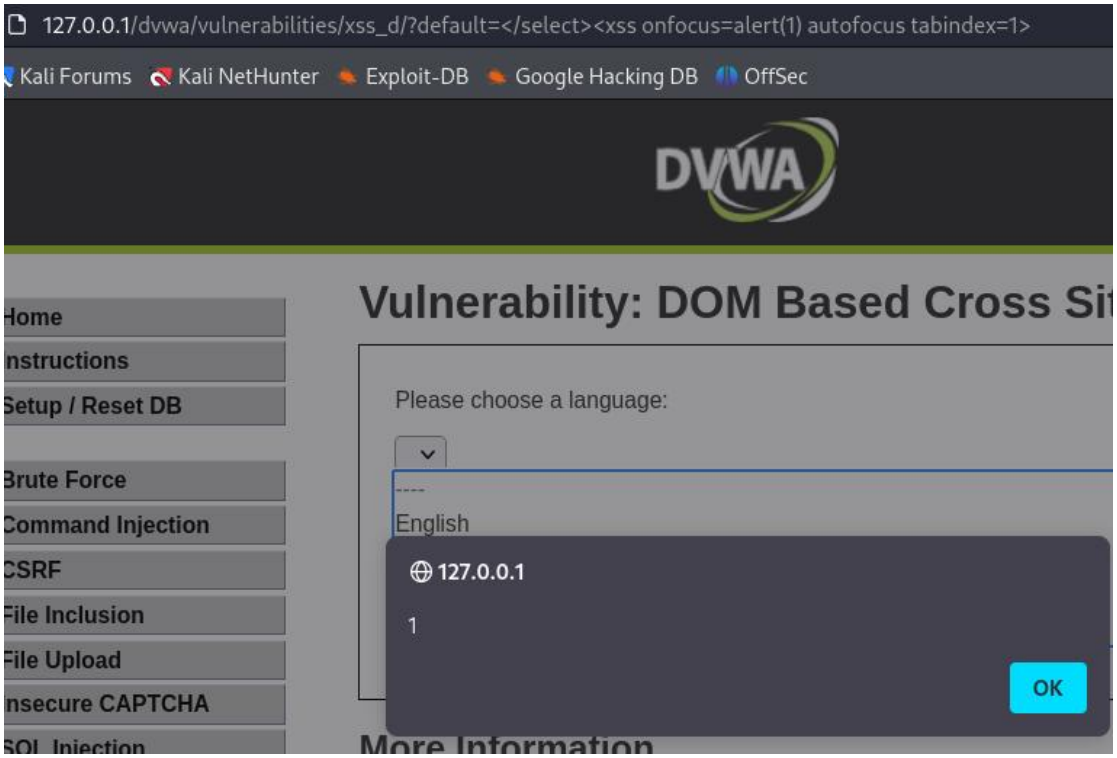
No protections

```
Payload:http://127.0.0.1/dvwa/vulnerabilities/xss_d/?default=<script>alert('XSS')</s
cript>
```

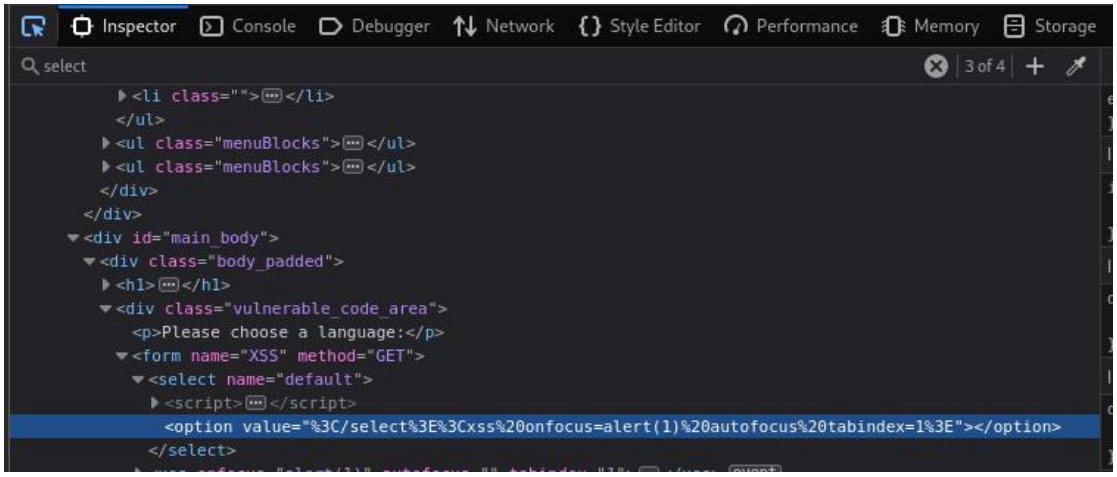
- **The payload explanation:**
  I- Browser loads the page
  II- JS reads `default` from the URL
  III- JS writes it into the DOM
  IV- Browser executes `<script>`

- **Mitigation:**
  I- Encode user input before inserting it into the DOM
  II- Implement Content Security Policy (CSP) to block inline scripts (`onfocus`, `onclick`)
  III- Input validation

II- Medium Security Level: Basic pattern-based input validation blocks `<script>` tags, but it is ineffective and allows attackers to bypass the control and execute JavaScript without using script tags.



type-0 XSS — medium sec lvl



The payload appears inside the `<select>` element, proving that user input is inserted into the DOM without proper sanitization

```
Payload:http://127.0.0.1/dvwa/vulnerabilities/xss_d/?default=</select><xss
onfocus=alert(1) autofocus tabindex=1>
```

- **The payload explanation:**
  I- Browser loads the page
  II- JS reads `default` from the URL
  III- JS writes the value directly into the DOM
  IV- HTML context is broken`</select>` closes the existing dropdown element, allowing injection of new HTML
  V- A malicious element is injected with a custom `<xss>` element is added with an `onfocus` event handler
  VI- The `autofocus` attribute forces focus when the page loads

- **Mitigation:**
  I- Encode user input before inserting it into DOM, so injected tags are rendered as text, not executed
  II- Avoid dangerous DOM sinks (`innerHTML`, `document.write()`), and use `textContent`/safe DOM APIs instead
  III- Implement Content Security Policy (CSP) to block inline scripts (`onfocus`, `onclick`)
  IV- Input validation

— — — —

Type-0 XSS successfully reveals session cookies (`security` and `PHPSESSID`) via `document.cookie`

```
Payload:http://127.0.0.1/dvwa/vulnerabilities/xss_d/#default=<script>alert(document.
cookie)</script>
```

- **The payload explanation:**
  I- Browser loads the page
  II- The client-side JS reads the `default` value from the URL fragment (`#`)
  III- The JS inserts this value directly into DOM without sanitization
  IV- The browser interprets the injected `<script>` tag and executes it, triggering `alert(document.cookie)`

- **Mitigation:**
  I- Encode user input before inserting it into the DOM
  II- Implement CSP
  IV- Input validation